

Các chương trước đã trình bày nền tảng lý thuyết và kết quả triển khai của hệ thống. Chương này tập trung phân tích các giải pháp và đóng góp nổi bật của đề án dưới góc nhìn thiết kế kỹ thuật. Trọng tâm không nằm ở việc liệt kê lại từng module, mà làm rõ các vấn đề khó đã gặp phải trong quá trình xây dựng trợ lý giáo dục dựa trên LLM và cách hệ thống giải quyết các vấn đề đó bằng kiến trúc có thể kiểm soát, dễ mở rộng và phù hợp với dữ liệu sách giáo khoa.

Các đóng góp chính gồm thiết kế điều phối request ở mức mã nguồn, cơ chế truy xuất thích nghi, nhận diện đa ý định, tách biệt pipeline quiz tương tác khỏi pipeline sinh nội dung dài, tổ chức content multi-agent cho slide và giáo án, chuẩn hóa giao tiếp nội bộ giữa các agent bằng A2A-lite, bổ sung HITL tại điểm quyết định quan trọng, và xây dựng các lớp kiểm tra chất lượng, session, trace phục vụ vận hành.

0.1 Điều phối request bằng thin orchestrator

0.1.1 Vấn đề cần giải quyết

Một hệ thống trợ lý giáo dục không chỉ nhận câu hỏi và gọi LLM để trả lời. Cùng một giao diện chat phải xử lý nhiều nhóm tác vụ khác nhau như hỏi đáp kiến thức, sinh câu hỏi luyện tập, chấm câu trả lời, ôn lại câu sai, tạo slide, tạo giáo án hoặc tiếp tục một pipeline đang chờ người dùng duyệt. Nếu đưa toàn bộ luồng xử lý vào một prompt lớn hoặc một agent tự quyết định mọi bước, hệ thống sẽ khó kiểm soát, khó gỡ lỗi và dễ gọi sai chức năng.

Vấn đề phức tạp hơn khi mỗi request cần mang theo nhiều loại trạng thái. Một câu hỏi có thể phụ thuộc lịch sử hội thoại, phạm vi sách/lớp người dùng chọn, chủ đề đã học trước đó, quiz state hoặc slide state đang chờ resume. Vì vậy, hệ thống cần một lớp điều phối rõ ràng để tách phần hiểu ngôn ngữ tự nhiên khỏi phần thực thi nghiệp vụ.

0.1.2 Giải pháp

Đề án thiết kế Orchestrator theo hướng thin controller. Orchestrator không chứa trực tiếp logic sinh câu hỏi, truy xuất tài liệu hoặc sinh slide, mà chủ yếu tạo `RequestContext`, phân tích ngữ cảnh, gọi `IntentRouter`, lập action plan và chuyển quyền thực thi cho `ExecutionDispatcher`. Cách làm này giữ Orchestrator ở vai trò điều phối, còn nghiệp vụ chi tiết được đóng gói trong các domain service hoặc handler.

`RequestContext` là state object đi xuyên qua toàn bộ vòng đời request. Nó lưu câu hỏi gốc, truy vấn đã làm giàu, danh sách query dùng cho RAG, intent, action plan, phạm vi sách/lớp, session và debug steps. Nhờ đó, các module không cần truyền nhiều biến rời rạc hoặc phụ thuộc vào global state. Khi cần kiểm tra một

lỗi, hệ thống có thể nhìn lại từng bước request đã đi qua.

Một điểm quan trọng là `ActionPlanner` hoạt động bằng rule-based logic thay vì gọi thêm LLM. LLM chỉ được dùng ở bước cần hiểu ngôn ngữ tự nhiên, còn ánh xạ từ intent sang action được giữ bằng mã nguồn tường minh. Điều này giảm độ trễ, tránh phát sinh nhánh xử lý ngoài kiểm soát và giúp việc thêm action mới có điểm mở rộng rõ ràng.

0.1.3 Đóng góp đạt được

Thiết kế này giải quyết được mâu thuẫn giữa tính linh hoạt của giao diện hội thoại và yêu cầu kiểm soát của hệ thống giáo dục. Người dùng vẫn có thể nhập yêu cầu tự nhiên, nhưng hệ thống không giao toàn bộ quyền điều phối cho LLM. Mỗi action được dispatch đến đúng service phụ trách, giúp kiến trúc dễ mở rộng hơn khi bổ sung chức năng mới.

Thin orchestrator cũng tạo nền tảng cho trace và debug. Mỗi request có request id, timestamp, debug steps và thông tin strategy RAG đã chọn. Đây là đóng góp thực tế quan trọng vì hệ thống LLM thường khó quan sát nếu chỉ nhìn đầu vào và đầu ra cuối cùng.

0.2 Truy xuất lại và Adaptive RAG

0.2.1 Vấn đề cần giải quyết

Dữ liệu sách giáo khoa có đặc điểm khác với dữ liệu web mở. Nhiều khái niệm quan trọng được trình bày ngắn, có thuật ngữ chuyên ngành, tên lệnh, tên bài học hoặc cấu trúc chương mục rõ ràng. Nếu chỉ dùng tìm kiếm ngữ nghĩa, hệ thống có thể bỏ sót các đoạn có từ khóa chính xác. Nếu chỉ dùng tìm kiếm từ khóa, hệ thống khó xử lý câu hỏi diễn đạt tự nhiên khác với văn bản trong sách.

Ngoài ra, không phải câu hỏi nào cũng cần cùng một cách truy xuất. Câu hỏi “giải thích khái niệm cơ sở dữ liệu” cần đoạn nội dung chi tiết, trong khi câu hỏi “lớp 12 có những chủ đề nào” cần tổng hợp cấu trúc chương trình. Một pipeline RAG cố định dễ hoặc lấy thiếu ngữ cảnh, hoặc tốn chi phí không cần thiết.

0.2.2 Giải pháp

Hệ thống sử dụng truy xuất lại gồm tìm kiếm từ khóa BM25-like, tìm kiếm ngữ nghĩa bằng embedding tiếng Việt, hợp nhất thứ hạng bằng RRF và xếp hạng lại bằng reranker. BM25 giúp giữ độ nhạy với thuật ngữ chính xác, còn embedding giúp bắt được các câu hỏi diễn đạt theo cách tự nhiên. RRF được dùng để hợp nhất hai danh sách kết quả mà không yêu cầu điểm số của chúng nằm trên cùng một thang đo.

Trên lớp truy xuất cơ sở, đề án xây dựng `AdaptiveRAGAgent`. Thành phần

này chọn strategy dựa trên `QueryClassifier` heuristic, không dùng LLM để ra quyết định. Bốn strategy chính gồm STANDARD, BROAD, CURRICULUM và HIERARCHICAL. STANDARD dùng cho truy vấn cụ thể nhưng thiếu phạm vi rõ. BROAD dùng cho câu hỏi tổng quan. CURRICULUM ưu tiên metadata chương trình học. HIERARCHICAL dùng khi có đủ tín hiệu lớp/chủ đề để truy xuất theo cấp bài học rồi mới đi vào chunk chi tiết.

Hệ thống còn bổ sung scope fallback. Khi phạm vi sách/lớp chỉ đến từ UI và không đủ chunk phù hợp, `RAGService` có thể mở rộng phạm vi tìm kiếm, đồng thời ghi lại `actual_scope`, `scope_fallback_notice` và debug step tương ứng. Nhờ vậy, hệ thống không trả lời rỗng quá sớm nhưng vẫn minh bạch về phạm vi dữ liệu đã dùng.

0.2.3 Đóng góp đạt được

Đóng góp ở đây không chỉ là ghép nhiều kỹ thuật retrieval, mà là đặt chúng trong một cơ chế chọn strategy theo loại câu hỏi. Cách làm này phù hợp với bài toán giáo dục vì câu hỏi của học sinh và giáo viên có mức độ cụ thể rất khác nhau. Adaptive RAG giúp hệ thống cân bằng giữa độ chính xác, độ bao phủ và chi phí truy xuất.

Việc `QueryClassifier` là heuristic cũng là một quyết định có chủ đích. Thay vì gọi LLM thêm một lượt để chọn chiến lược truy xuất, hệ thống giữ bước này nhanh, có thể kiểm thử và có thể giải thích. Đây là điểm quan trọng trong một hệ thống cần ổn định hơn là thể hiện khả năng suy luận tự do.

0.3 Multi-intent routing và action planning

0.3.1 Vấn đề cần giải quyết

Trong giao diện hội thoại, người dùng không luôn đưa ra một yêu cầu đơn lẻ. Một câu có thể chứa nhiều ý định, ví dụ yêu cầu tạo slide rồi thêm câu hỏi luyện tập, hoặc vừa giải thích kiến thức vừa sinh bài tập. Nếu hệ thống chỉ chọn một intent duy nhất, một phần yêu cầu sẽ bị bỏ qua. Ngược lại, nếu để LLM tự thực thi toàn bộ chuỗi hành động, hệ thống dễ sinh ra luồng không nhất quán với các service đang có.

0.3.2 Giải pháp

`IntentRouter.detect_multi()` phân tích tối đa ba intent trong một câu hỏi. Mỗi intent gồm nhóm ý định chính, loại nhiệm vụ, chủ đề, lớp, bộ sách và một số metadata cần thiết. Sau đó, `ActionPlanner.plan_all()` chuyển danh sách intent thành danh sách action cụ thể như sinh quiz, sinh slide, sinh giáo án, chấm câu trả lời, ôn câu sai hoặc giải thích khái niệm.

Việc giới hạn số intent là một quyết định thực tế. Nó đủ để xử lý các câu lệnh ghép thường gặp, nhưng tránh để một câu nhập mơ hồ sinh ra quá nhiều action và làm phình state. Sau bước lập kế hoạch, `ExecutionDispatcher` chịu trách nhiệm gọi đúng service hoặc handler theo action đã xác định.

0.3.3 Đóng góp đạt được

Cơ chế này giúp hệ thống xử lý yêu cầu tự nhiên tốt hơn mà vẫn giữ được tính quyết định của backend. LLM được dùng để hiểu câu nói của người dùng, nhưng kế hoạch thực thi cuối cùng được đưa về cấu trúc action rõ ràng. Điều này tạo ra một ranh giới cần thiết giữa năng lực ngôn ngữ của LLM và logic nghiệp vụ của phần mềm.

0.4 Tách quiz standalone khỏi content assessment

0.4.1 Vấn đề cần giải quyết

Sinh câu hỏi luyện tập có thể xuất hiện trong hai bối cảnh khác nhau. Bối cảnh thứ nhất là quiz standalone, nơi người dùng tạo bộ câu hỏi, trả lời, được chấm điểm, xem lại câu sai và theo dõi tiến độ. Bối cảnh thứ hai là assessment nhúng trong slide hoặc giáo án, nơi câu hỏi chỉ là một artifact phụ để củng cố bài giảng.

Nếu gộp hai bối cảnh này vào cùng một agent hoặc cùng một pipeline, hệ thống sẽ khó quản lý state. Quiz standalone cần lưu từng lượt làm bài, từng câu hỏi, câu trả lời của người dùng, số lần thử và kết quả chấm. Assessment trong slide lại chủ yếu cần nội dung phù hợp với bài học và format trình bày.

0.4.2 Giải pháp

Đồ án giữ `QuizService` là một service độc lập cho quiz tương tác. Service này quản lý vòng đời sinh câu hỏi, kiểm tra chất lượng, lưu `QuizRound`, lưu `QuestionRecord`, chấm câu trả lời, review câu sai và thống kê tiến độ. Các handler câu hỏi như trắc nghiệm, tự luận, điền khuyết và đúng/sai được giữ trong pipeline quiz riêng.

Trong content multi-agent pipeline, `ContentAssessmentAgent` chỉ sinh assessment nhúng cho slide hoặc giáo án. Agent này không thay thế `QuizService` và không quản lý state làm bài của học sinh. Cách tách này giúp tránh nhầm lẫn giữa một bài kiểm tra tương tác và một phần hoạt động trong tài liệu giảng dạy.

0.4.3 Đóng góp đạt được

Đây là một đóng góp kiến trúc quan trọng vì nó giữ đúng ranh giới nghiệp vụ. Quiz standalone có thể phát triển thêm các chức năng học tập cá nhân hóa mà không ảnh hưởng đến pipeline sinh slide. Ngược lại, pipeline sinh slide/giáo án có thể tối ưu cho artifact dài mà không phải mang theo toàn bộ logic chấm điểm và

theo dõi attempt.

0.5 Content multi-agent cho slide và giáo án

0.5.1 Vấn đề cần giải quyết

Tạo slide và giáo án là tác vụ dài hơn nhiều so với hỏi đáp hoặc sinh một bộ câu hỏi ngắn. Hệ thống phải xác định mục tiêu học tập, lập dàn ý, viết nội dung chi tiết, gợi ý media, thêm assessment, ghép kết quả và kiểm tra chất lượng. Nếu giao toàn bộ tác vụ này cho một prompt duy nhất, đầu ra dễ thiếu cấu trúc, bỏ sót phần quan trọng hoặc khó sửa khi người dùng muốn thay đổi dàn ý.

0.5.2 Giải pháp

Đồ án dùng LangGraph cho phạm vi content pipeline của slide và giáo án. Ở cấp hệ thống, Orchestrator vẫn là controller mã nguồn; LangGraph chỉ điều phối luồng sinh nội dung dài. ContentSupervisor gọi các tool adapter tương ứng với các specialist agent như PedagogyPlannerAgent, ContentDraftingAgent, MediaResearchAgent, ContentAssessmentAgent và QualityReviewerAgent.

Các agent không trao đổi tự do mà đi qua contract A2A-lite. Supervisor tạo AgentTask, specialist agent trả AgentTaskResult. Task chứa mục tiêu, đầu vào, ràng buộc, context refs và artifact mong đợi. Kết quả chứa trạng thái, artifact, confidence, warnings, used tools và latency. Contract này giúp mỗi kết quả trung gian có chủ sở hữu rõ ràng và dễ ghi debug.

Sau khi các artifact được tạo, merge_results ghép outline, content, media và assessment thành kết quả thống nhất. Thành phần merge là service/tool quyết định, không phải agent. Sau đó, QualityReviewerAgent đánh giá kết quả cuối và reflection_decision_node quyết định approve, revise hoặc block theo số vòng reflection cho phép.

0.5.3 Đóng góp đạt được

Đóng góp chính của phần này là tổ chức tác vụ sinh nội dung dài thành các bước có artifact rõ ràng. Hệ thống không chỉ sinh một đoạn văn lớn, mà có dàn ý, nội dung, media metadata, assessment và review. Điều này giúp slide và giáo án dễ kiểm soát hơn, đồng thời chuẩn bị nền tảng để sau này xuất sang PPTX, PDF hoặc HTML.

A2A-lite cũng là một lựa chọn phù hợp với quy mô hiện tại. Hệ thống chưa cần runtime distributed agent đầy đủ, nhưng đã có message envelope đủ rõ để quản lý task, artifact và metadata thực thi. Cách làm này giảm độ phức tạp vận hành nhưng vẫn giữ đường nâng cấp trong tương lai.

0.6 Human-in-the-loop tại bước duyệt dàn ý

0.6.1 Vấn đề cần giải quyết

Trong sinh slide và giáo án, dàn ý là quyết định ảnh hưởng đến toàn bộ artifact phía sau. Nếu dàn ý sai trọng tâm, thiếu mục hoặc chia bài không hợp lý, các bước sinh nội dung, media và assessment sau đó đều bị ảnh hưởng. Sửa ở cuối pipeline thường tốn chi phí hơn nhiều so với sửa ngay sau khi có outline.

0.6.2 Giải pháp

Hệ thống đặt HITL tại bước outline review. Sau khi agent lập dàn ý, graph có thể interrupt để người dùng duyệt hoặc gửi feedback chỉnh sửa. SlideService lưu các thông tin resume như thread id, graph config, trạng thái interrupt và task type vào slide state. Khi người dùng xác nhận hoặc góp ý, pipeline resume từ điểm đã dừng thay vì chạy lại toàn bộ request từ đầu.

0.6.3 Đóng góp đạt được

HITL giúp đưa quyền kiểm soát vào đúng điểm có ảnh hưởng lớn nhất. Người dùng, đặc biệt là giáo viên, có thể điều chỉnh cấu trúc bài giảng trước khi hệ thống sinh nội dung chi tiết. Cách làm này phù hợp với đặc thù giáo dục, nơi người dùng thường muốn kiểm soát mục tiêu và mạch bài học thay vì chỉ nhận một kết quả hoàn chỉnh.

0.7 Quality path, session và trace

0.7.1 Vấn đề cần giải quyết

Một hệ thống LLM phục vụ giáo dục cần nhiều lớp kiểm soát hơn một chatbot thông thường. Câu hỏi sinh ra cần đúng format và bám ngữ cảnh. Câu trả lời cần lưu được theo phiên để người dùng hỏi tiếp. Các lỗi cần có trace để biết hệ thống đã chọn intent, action và strategy RAG nào.

0.7.2 Giải pháp

Đồ án xây dựng hai quality path riêng. Với quiz standalone, QualityReviewer kiểm tra chất lượng bộ câu hỏi và QuestionValidator kiểm tra từng câu hỏi trước khi lưu vào session. Nếu cần, handler có thể được gọi lại với revision instruction. Với slide và giáo án, kết quả sau merge được đưa qua QualityReviewerAgent; graph có thể approve, revise một phần hoặc block nếu chất lượng chưa đạt.

Session được lưu bằng SessionStore dưới dạng JSON. Session không chỉ lưu lịch sử chat mà còn lưu quiz state và slide state, cho phép các yêu cầu nối tiếp như “chấm câu 2”, “ôn câu sai” hoặc “ok, duyệt dàn ý”. Song song với session, TraceService và RequestContext.add_debug_step() ghi lại

các bước xử lý quan trọng để hỗ trợ kiểm thử và gỡ lỗi.

0.7.3 Đóng góp đạt được

Các lớp quality, session và trace làm cho hệ thống có khả năng vận hành thực tế hơn. Hệ thống không chỉ tạo được câu trả lời, mà còn quản lý được vòng đời học tập của người dùng và cung cấp dữ liệu quan sát khi kết quả chưa đạt kỳ vọng. Đây là nền tảng cần thiết để tiếp tục cải thiện chất lượng truy xuất, đánh giá và cá nhân hóa trong các phiên bản sau.

0.8 Tool layer như điểm mở rộng có kiểm soát

0.8.1 Vấn đề cần giải quyết

Một số năng lực của hệ thống không nên được nhúng trực tiếp vào prompt hoặc agent, ví dụ truy xuất tri thức, định dạng context, tìm kiếm media, kiểm tra URL hoặc xuất artifact. Nếu mỗi nơi gọi tool theo một kiểu riêng, hệ thống sẽ khó mở rộng và khó thay thế implementation.

0.8.2 Giải pháp

Đồ án xây dựng một MCP-like tool layer in-process. Lớp này gồm protocol request/response, client, server và registry tool. Các tool như knowledge retrieval hoặc content formatting có thể được gọi qua interface thống nhất. Hiện tại, lớp này chủ yếu đóng vai trò chuẩn hóa tool call nội bộ và làm extension point; chưa được mô tả như một MCP phân tán hoàn chỉnh.

0.8.3 Đóng góp đạt được

Tool layer giúp tách phần năng lực mở rộng khỏi core orchestration. Khi cần bổ sung media search thật, URL validation, source attribution hoặc export artifact, hệ thống đã có vị trí phù hợp để tích hợp mà không làm thay đổi sâu các service hiện có. Đây là một đóng góp về khả năng mở rộng hơn là một tính năng hoàn chỉnh ở thời điểm hiện tại.

0.9 Kết chương

Chương này đã hệ thống hóa các giải pháp và đóng góp chính của đồ án. Các đóng góp không nằm ở một thuật toán riêng lẻ, mà ở cách kết hợp LLM với các lớp điều phối, truy xuất, kiểm tra chất lượng, quản lý trạng thái và quan sát hệ thống. Thiết kế này giúp Educational Chatbot tránh phụ thuộc hoàn toàn vào khả năng sinh tự do của LLM, đồng thời giữ được tính mở rộng cho các chức năng giáo dục như quiz, slide và giáo án.

Những đóng góp trên cũng cho thấy các khó khăn chính của đề tài đã được xử lý theo hướng thực dụng: truy xuất phải thích nghi với loại câu hỏi, điều phối phải kiểm soát được nhiều intent, quiz tương tác phải tách khỏi artifact generation,

multi-agent phải có contract rõ ràng, và người dùng cần được đưa vào vòng duyệt ở các điểm quyết định quan trọng.